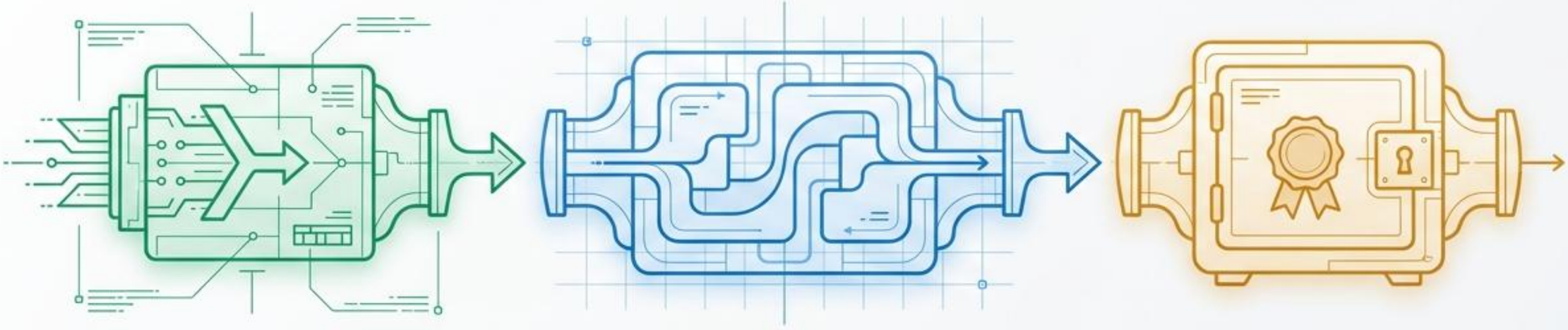
모던 자바 파이프라인: 더 짧고, 더 안전하고, 더 우아하게

비전공자와 주니어 개발자를 위한 람다, 스트림, 그리고 Optional 완벽 가이드



() → {} **람다식:** 코드 다이어트

스트림: 조립 라인

[?] **Optional:** 안전 포장재

NotebookLM

우리가 모던 자바 문법을 배워야 하는 이유

```
for (String> i = 0; data.length; i++) {
    (String> i == 0; data.length; i++) {
    if (item.item != null) {
        if (item.item -> item != null) {
            return item;
        }
        else if(item != null) {
            if (!item.isAvailable()) {
                .map(String::toUpperCase);
            }
        }
        return results;
    }
}
```

과거의 자바: 장황하고 반복적인 코드

비전공자 핵심 요약:
어떻게(How) 할 것인가를 일일이 지시하는 대신, 무엇을(What) 할 것인지만 선언하세요. 코드는 짧아지고, 유지보수는 쉬워집니다.

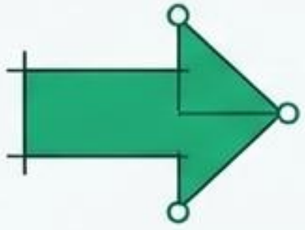
```
List<String> results = data.stream()
    .filter(item -> item != null && item.isAvailable())
    .map(String::toUpperCase)
    .collect(Collectors.toList());
```

모던 자바: 의도가 한눈에 보이는 코드

NotebookLM

람다식(Lambda): 이름 없는 함수, 궁극의 코드 요약본

```
new Runnable() {
    public void run() {
        ...
    }
}
```



```
() -> System.out.println("Hello!");
```

화살표(전달)

(매개변수) -> { 실행문 }

입력값

실행 로직

매개변수가 하나거나, 실행문이 한 줄이면 괄호 () 와 { } 도 생략 가능합니다.

NotebookLM

참조(Reference): 직접 만들지 말고, 있는 걸 쓰세요

구분	멘탈 모델	람다식 (Before)	메서드 참조 (After)
정적/인스턴스 메서드	이 메서드 그대로 쓸게요	`s -> System.out.println(s)`	`System.out::println`
객체 생성자	이 설계도로 새 객체 만들어주세요	`n -> new Fruit(n)`	`Fruit::new`

비전공자 핵심 요약:
람다식이 '이렇게 해주세요'라면, 메서드 참조는 '이거 이미 만들어진 거 쓰세요' 입니다.
:: 기호를 보면 '누구의 무엇을 쓴다'고 읽으세요.

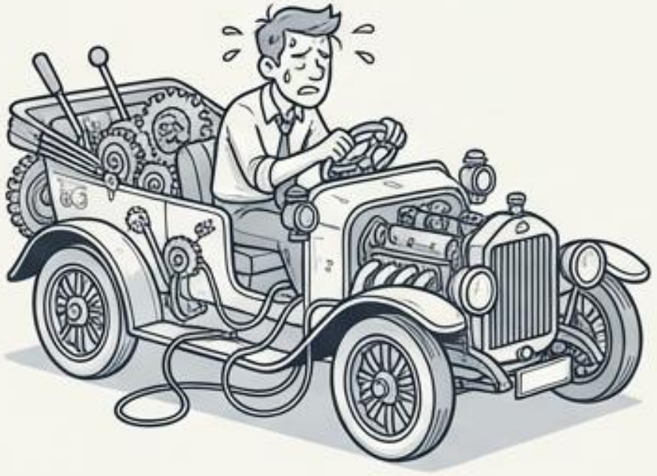
NotebookLM

스트림(Stream): 데이터 처리의 자동화 공장

Who Drives? (운전자는 누구?)

외부 반복자 - External Iteration

개발자가 직접 (for, while)



컬렉션에서 데이터를 하나씩 꺼내와서 직접 지시.

```
for(String s : list) { ... }
```

내부 반복자 - Internal Iteration

스트림이 알아서 (stream.forEach)

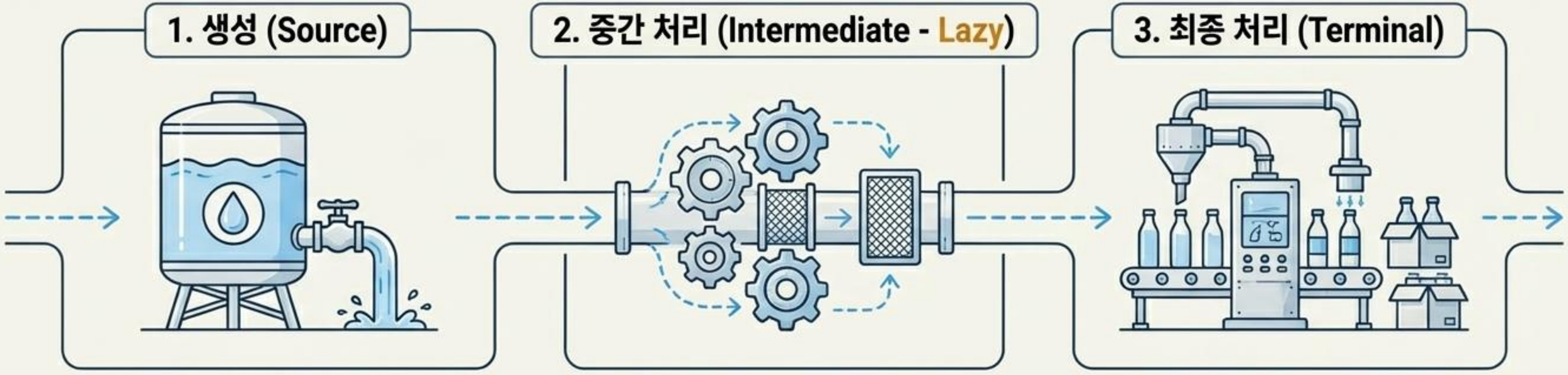
스트림 엔진에 무엇을 할지만 넘기면, 알아서 반복 처리.

```
list.stream().forEach(...)
```

비전공자 핵심 요약: 스트림은 물줄기입니다. 물줄기처럼 데이터가 흘러가면서 필요한 작업만 걸러내고 변형합니다. 반복문 작성의 고통에서 벗어나세요.

NotebookLM

스트림 파이프라인의 3단계 생명주기



데이터를 스트림으로 변환.
(컬렉션, 배열, 파일)

```
list.stream() /
Arrays.stream(arr)
```

흐름을 변경하거나 가공.
지연(Lazy) 실행되어
최종 처리 때 한 번에 수행됨.

```
.filter()
.map()
.sorted()
```

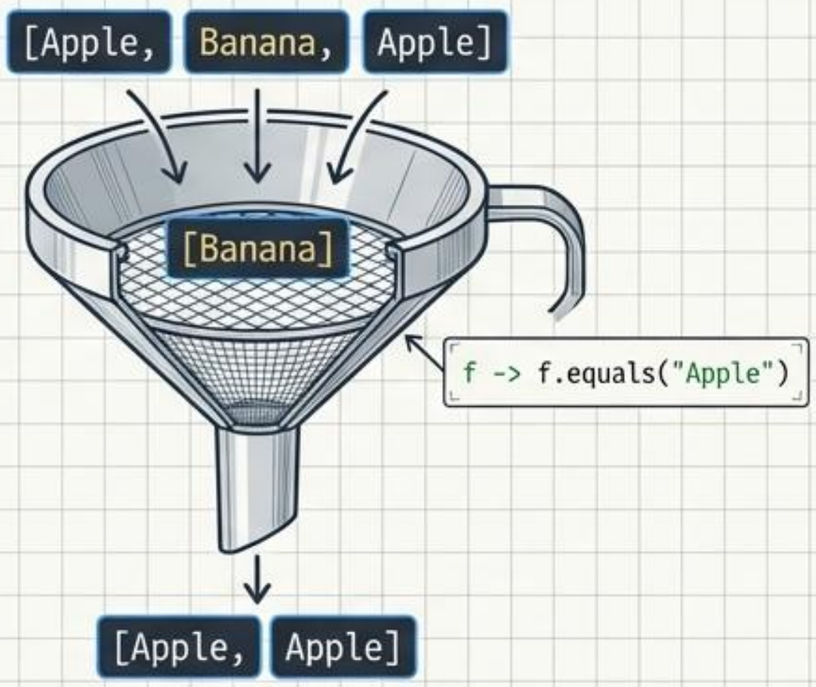
파이프라인의 끝. 결과를
출력하거나 모아서 스트림을 닫음.
호출해야만 공장이 가동됨!

```
.forEach()
.collect()
.count()
```

NotebookLM

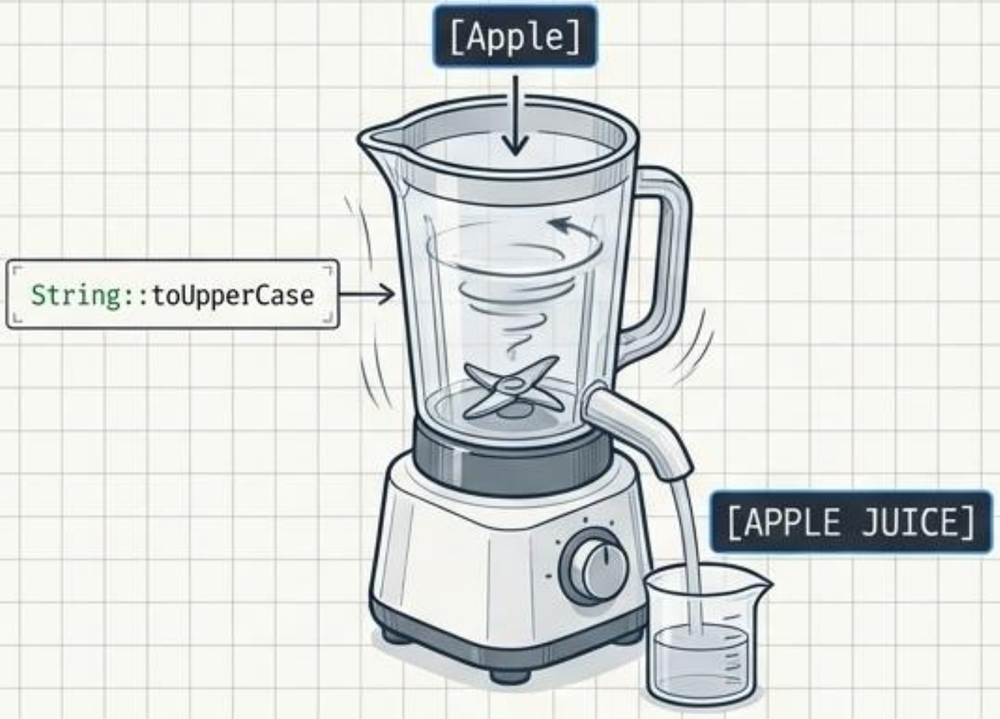
핵심 중간 처리: 걸러내기(Filter)와 변형하기(Map)

Filter - The Mesh




```
.filter(n -> n % 2 == 0) // 짝수만 통과
```

Map - The Blender




```
.map(n -> n * n) // 모든 요소를 제곱으로 변환
```

기억하세요: Filter는 데이터의 갯수가 줄어들 수 있지만 형태는 그대로입니다. Map은 데이터의 갯수는 같지만 형태가 완전히 바뀝니다.

NotebookLM

스트림 오퍼레이션 완벽 분류표 (Cheat Sheet)

목적 (Purpose)	메서드 (Method)	분류 (Type)	설명 (Description)
정렬 (Sort)	`sorted()`	중간(Intermediate)	요소를 오름차순/내림차순으로 정렬
루핑 (Loop)	`forEach()`	최종(Terminal)	각 요소를 하나씩 꺼내어 작업 실행
검사 (Match)	`allMatch()`, `anyMatch()`	최종(Terminal)	조건 만족 여부를 boolean으로 반환 (모두/하나라도)
통계 (Aggregate)	`sum()`, `max()`, `reduce()`	최종(Terminal)	숫자 합계, 최대값, 혹은 사용자 정의 누적 계산
수집 (Collect)	`collect()`	최종(Terminal)	처리된 스트림을 List, String 등으로 다시 묶음

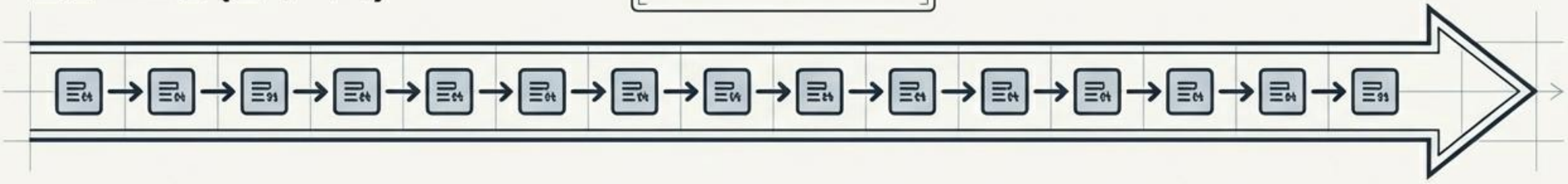
비전공자 핵심 요약: 파이프라인의 마지막에는 반드시 '**최종 처리**' 메서드가 와야 합니다. 최종 메서드가 없으면 스트림은 한 방울의 물도 흘려보내지 않습니다 (**지연 실행**).

NotebookLM

보너스 트랙: 대량 데이터의 마법, 병렬 스트림

일반 스트림 (순차 처리)

`.stream().sum()`



병렬 스트림 (동시 처리)

`.parallelStream().sum()`

⚠ **주의:** 병렬 처리는 속도를 획기적으로 높일 수 있지만, forEach 출력 순서가 매번 달라질 수 있습니다. 순서가 중요하지 않은 대규모 계산에만 사용하세요.

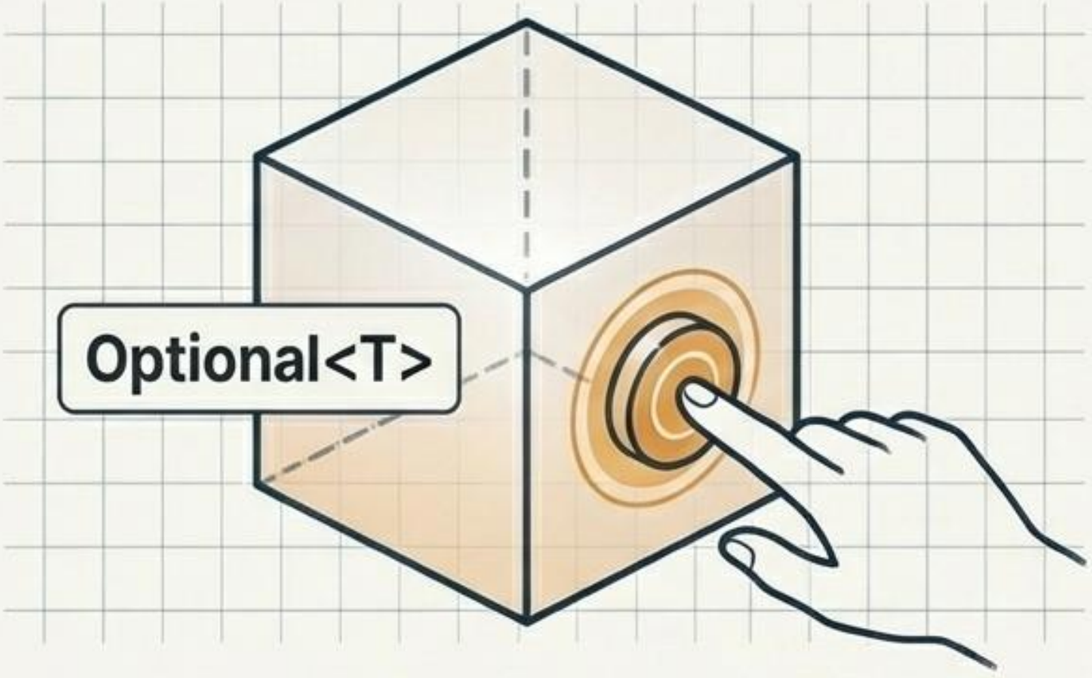
NotebookLM

NullPointerException의 공포, 그리고 Optional의 등장

과거의 방식

값이 없는 줄 모르고 접근 (시스템 크래시)

Optional 방식



상자 외부의 안전한 버튼을 통해 상호작용

비전공자 핵심 요약: Optional은 '값이 있을 수도, 없을 수도 있는 상자' 입니다.
값이 없는 상태(null)를 상자로 한 번 감싸서, 프로그램이 갑자기 죽는 것을 완벽하게 예방합니다.

NotebookLM

안전하게 상자를 여는 3가지 프로토콜

기본값 보장 (orElse)

값이 없으면 준비해둔 '기본값'을
대신 사용합니다.

```
opt.orElse("Default");
```

있을 때만 실행 (ifPresent)

값이 존재할 때만 람다식을 실행합니다.
없으면 조용히 넘어갑니다.

```
opt.ifPresent(System.out::println);
```

당당한 예외 발생 (orElseThrow)

값이 반드시 있어야 하는 로직에서,
없을 경우 명시적으로 에러를 던집니다.

```
opt.orElseThrow(Exception::new);
```

NotebookLM

모든 것을 하나로: 모던 자바 마스터피스

파이프라인 개통
(컬렉션을 스트림으로)

람다식
(간결한 조건 지시)

메서드 참조
(이미 있는 기능 재활용)

안전 포장재
(결과가 없을 때의 대비책)

```
Optional<String> topFruit = fruits.stream()
        .filter(f -> f.length() > 5)
        .map(String::toUpperCase)
        .sorted()
        .findFirst();

topFruit.ifPresent(System.out::println);
```

최종 처리
(스트림 닫기, 결과 도출)

무엇을 할지 선언하고, 물줄기처럼 흘려보내며, 빈 상자라도 안전하게 다룹니다.
이것이 모던 자바의 우아함입니다.

NotebookLM